ISYE 7406 Final Report

Room Occupancy Estimation Using Data Mining and Statistical Learning

Project Team 054

Yi Liu Gul (343), Qi Pan (059)

yliu330@gatech.edu, qpan40@gatech.edu

November 15, 2025

Abstract

This project develops a comprehensive modeling pipeline for multi-class room occupancy prediction using environmental sensor data. We evaluate classifiers including LDA, QDA, Naive Bayes, Logistic Regression, KNN, and SVM. Each applied across three feature pipelines: full features, stepAIC-selected features, and PCA-transformed predictors with model-specific optimal principal components. In addition, two ensemble methods, Random Forest and Gradient Boosting Machines (GBM), are tuned using systematic hyperparameter searches.

Across 30+ trained models, results show consistently high predictive accuracy, with the top-performing Random Forest model achieving 0.9965 test accuracy, closely followed by tuned SVM, Logistic Regression, KNN, and GBM models (0.994–0.996). In contrast, QDA and LDA perform moderately well, and Naive Bayes yields the lowest accuracy among all models. Feature selection produces models comparable to full models, while PCA does not provide meaningful accuracy gains. Random Forest feature importance indicates that illumination and CO_2-related variables, including the CO_2 slope, are influential predictors. These findings suggest that environmental sensing alone is highly effective for occupancy estimation, and ensemble methods provide the most robust performance.

1 Introduction

Occupancy estimation is a core component of intelligent building automation, enabling adaptive HVAC and lighting control, improving facility utilization, and supporting safety and robotics applications. Traditional sensing approaches, such as cameras or wearable devices, can achieve high accuracy but raise concerns regarding privacy, hardware cost, and deployment scalability. In contrast, environmental sensor-based methods rely on ambient signals such as temperature, light, sound, and CO_2 concentration, offering a low-cost and privacy-preserving alternative suitable for offices, schools, hospitals, and residential environments [4].

Prior work has demonstrated that environmental sensing paired with statistical learning can achieve strong predictive performance. For example, Candanedo and Feldheim [1] achieved 95–99% accuracy using light, temperature, humidity, and CO_2 data. Additional studies have highlighted the importance of CO_2 dynamics for occupancy modeling, particularly where delayed indoor CO_2 accumulation affects real-time prediction [2]. While many studies focus on feature selection or specific modeling approaches, fewer works systematically compare classical classification models, ensemble methods, and dimensionality-reduction strategies in the context of real-time and edge-deployable occupancy estimation.

This project investigates machine learning techniques for estimating room occupancy counts (0-3 people) using environmental sensor data from a controlled indoor setting. We compare multiple supervised learning algorithms, including linear and nonlinear classifiers, tree-based ensembles, and gradient boosting. We further evaluate the role of principal component analysis (PCA) and stepwise feature selection in addressing feature correlation, model complexity, and computational efficiency. The final report presents complete performance results across all modeling pipelines, analyzes feature importance, including CO_2 slope effects, and

provides a comprehensive comparison of accuracy, robustness, and interpretability among competing approaches.

2 Problem Statement and Data Source

2.1 Problem Statement

The goal of this study is to develop supervised learning models for estimating room occupancy counts using non-intrusive environmental sensor data. The primary research questions include:

- How well can environmental data predict occupancy?
- Which features (such as CO_2 or light) are the most important?
- Does PCA help improving models' performance?
- Does incorporating CO_2 dynamics (e.g., CO_2 Slope) improve predictions by capturing delayed accumulation patterns?

To address these questions, we evaluate multiple models, including Logistic Regression, Linear Discriminant Analysis (LDA), Quadratic Discriminant Analysis (QDA), Naive Bayes, k-Nearest Neighbors (KNN), Support Vector Machines (SVM), Random Forests, and Gradient Boosting Machines (GBM). In addition to full-feature models, we also examine a feature-selection pipeline using stepwise AIC and compare performance with PCA-transformed features to assess trade-offs between interpretability, accuracy, and computational efficiency for potential real-time deployment.

2.2 Data Source

The dataset is publicly available from the UCI Machine Learning Repository [3]. It contains five environmental variables: temperature, light, sound, CO_2, and digital passive infrared (PIR). The response variable indicates the number of occupants in the room, ranging from 0 to 3 people. Observations were recorded every 30 seconds with the number of occupants present. The data were collected in a controlled office environment, a 6-meter by 4.6-meter room equipped with seven sensor nodes.

A preliminary data review confirmed no missing values, notable class imbalance toward unoccupied samples, and strong correlations among certain sensor groups (e.g., light and temperature channels). These properties suggest both feature standardization and exploration of PCA for models sensitive to correlated predictors.

Figure 1 displays the Pearson correlation heatmap for all continuous predictors. Strong positive correlations are observed within sensor groups, particularly among the four temperature sensors and four light sensors. For example, temperature readings S1-S4 are highly correlated, as are light readings, suggesting substantial redundancy among these variables. CO_2 and CO_2 slope exhibit moderate correlation, consistent with expected accumulation behavior. PIR motion sensors show weak correlations with other features, reflecting the fundamentally different nature of binary motion signals.

These findings support two subsequent methodological decisions: (1) standardization of predictors due to heterogeneous scales, and (2) inclusion of feature selection and PCA experi-

ments to reduce redundancy among correlated variables and assess whether dimensionality reduction improves model stability and performance.

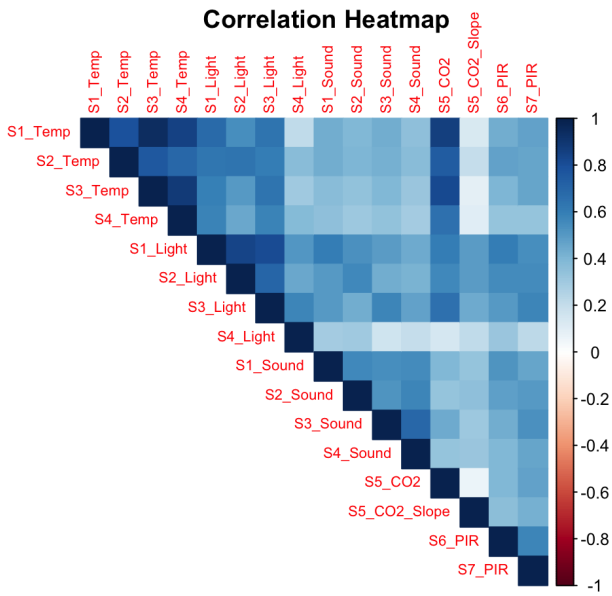

Figure 1: Correlation Heatmap

To examine how sensor measurements vary across occupancy classes, Figure 2 shows boxplots for each predictor stratified by occupancy count. Light intensity increases clearly with higher occupancy, consistent with typical lighting use when people are present. CO_2 and CO_2 slope also rise with occupancy, reflecting expected accumulation patterns. Temperature shows only a mild upward shift across classes, likely due to limited HVAC activity during data collection. Sound and PIR activation exhibit higher values in occupied states, although with greater variability, suggesting they serve as complementary rather than primary predictors.

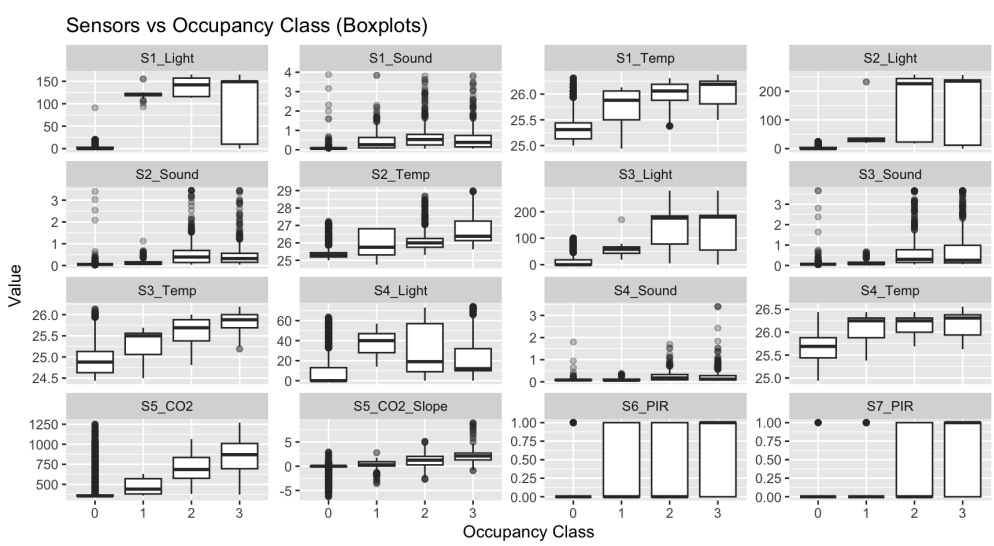

Figure 2: Sensor Value Distributions Across Occupancy Classes (0-3)

Figure 3 presents scatter plots illustrating the relationship between each predictor and numeric occupancy. Higher occupancy levels concentrate at elevated light and CO_2 values, underscoring their importance. Temperature shows modest separation across classes, while PIR features display near-binary shifts, acting as reliable presence indicators rather than fine-grained count predictors. These patterns guide the later modeling work, suggesting that:

- Light and CO_2-related features are likely to be highly informative.
- PIR activation is a reliable binary presence signal but not a strong discriminator of higher occupancy counts.
- Multicollinearity in temperature and light variables justifies PCA exploration.

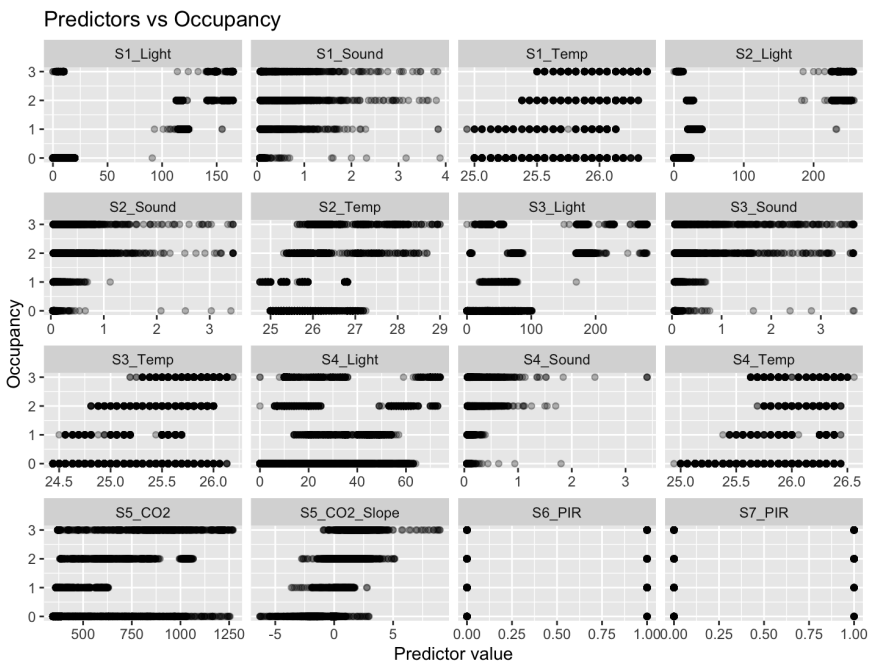

Figure 3: Predictor Values vs. Occupancy

The EDA indicates several patterns that directly motivate our modeling strategy. Strong correlations within the temperature and light groups suggest redundancy in the predictor space, supporting the use of PCA to reduce dimensionality and incorporating feature selection to identify the most informative predictors and reduce noise before training linear or distance-based classifiers.

3 Proposed Methodology

The goal of this project is to systematically evaluate supervised learning methods for predicting room occupancy counts (0-3 people) using environmental sensor data. Our approach integrates structured preprocessing, exploratory data analysis, feature selection, supervised modeling, PCA-based dimensionality reduction, and model evaluation.

3.1 Data Preprocessing

Consistent with best practices in prior occupancy detection research [1, 4, 2], preprocessing begins by removing the `Date` and `Time` fields, as this work focuses on real-time classification rather than time-series forecasting. The response variable `Room_Occupancy_Count` is recast as a factor with four classes. Continuous sensor variables, including temperature, light, sound, and CO_2-related readings, are standardized to zero mean and unit variance, while binary PIR motion indicators are retained without scaling. Data are split into 80% training and 20% testing using a fixed random seed to ensure consistent evaluation across all models and pipelines.

3.2 Exploratory Data Analysis

EDA includes summary statistics, correlation analysis, boxplots, and scatter visualizations. Sensor variables exhibit heterogeneous measurement scales, confirming the need for standardization. Strong within-group correlations, particularly among temperature and light sensors, suggest potential multicollinearity. These findings motivate two methodological components integrated in this project: (1) logistic regression-based StepAIC feature selection to reduce redundant predictors, and (2) PCA experiments to evaluate whether dimensionality reduction improves linear and distance-based models.

3.3 Supervised Learning Models

We evaluate a suite of supervised learning models commonly applied in environmental sensing and occupancy prediction, including:

- Logistic Regression
- Linear Discriminant Analysis (LDA)
- Quadratic Discriminant Analysis (QDA)
- Naive Bayes
- k-Nearest Neighbors (KNN)
- Support Vector Machine (SVM with RBF kernel)
- Random Forest
- Gradient Boosting Machine (GBM)

Logistic Regression, LDA, QDA, and Naive Bayes models are trained directly on the standardized features. Because KNN performance strongly depends on the choice of k, we perform Monte Carlo cross-validation with 60 randomized train/validation splits and select $k \in \{1, 3, 5, 7, 9\}$ based on minimum mean validation error. For SVM, we conduct a grid search over C and γ (powers of 2) using `tune.svm`.

Tree-based ensemble models, Random Forest and GBM, are tuned via grid search. Random Forest tuning covers `ntree`, `mtry`, and `nodesize`, selecting the final configuration based on Out-of-Bag (OOB) error. Gradient boosting hyperparameters include `shrinkage`, `interaction.depth`, and number of trees selected via 10-fold cross-validation.

3.4 Feature Selection via StepAIC

To investigate the value of a reduced feature space, we apply bidirectional StepAIC on a full multinomial logistic regression model using only the training set. The predictors retained in the final AIC-optimal model define the feature-selected pipeline. Six classical models, including Logistic Regression, LDA, QDA, Naive Bayes, KNN, and SVM, are re-trained on these selected variables and evaluated on the same test set.

3.5 Dimensionality Reduction with PCA

To evaluate the effect of correlated environmental predictors, Principal Component Analysis (PCA) is applied to the standardized continuous training variables. PCA transformation is then applied to the test data using the same rotation matrix. For LDA, QDA, Naive Bayes, Logistic Regression, and SVM, we select the number of principal components that minimizes validation error. For KNN, we jointly search over principal components and k values, selecting the (PC, k) pair achieving the lowest validation loss. Final models are then retrained using the full training set projected onto the chosen PCA space. Ensemble methods, such as Random Forest and GBM, are not combined with PCA, as they inherently handle correlated predictors and provide interpretable feature importance measures.

3.6 Feature Engineering for CO_2 Dynamics

Prior research emphasizes delayed CO_2 accumulation in enclosed environments [2]. Our initial plan included generating moving-average features to capture slower concentration buildup effects. However, we observed that the dataset already includes a derived feature, `CO2 Slope`, representing the instantaneous rate of CO_2 change. Exploratory analysis showed that this slope variable carries meaningful temporal information and exhibits stronger separation across occupancy levels than the raw CO_2 measurement. Given its effectiveness and to maintain parsimony, we decided not to engineer additional moving-average features.

3.7 Evaluation Framework

All models are evaluated on the held-out test set. We compute accuracy, micro/macro-averaged precision, recall, F1 scores, and specificity using a unified custom metric function. Confusion matrices and variable importance profiles (from Random Forest) support model interpretability. The methodology is designed to test two hypotheses: (1) that CO_2-related features, including the provided CO_2 slope variable, improve classification by capturing accumulation dynamics; and (2) that PCA or feature selection may benefit linear and distance-based classifiers in the presence of correlated environmental signals.

4 Analysis and Results

This section presents model performance results, feature selection outcomes, PCA configuration choices, and feature importance analyses from all experiments. All results are derived from the 80/20 train-test split and evaluated on the held-out test set. Metrics include accuracy, micro- and macro-averaged precision, recall, F1 score, and specificity.

4.1 Feature Selection and Its Impact

To assess whether a more compact predictor set could improve classification accuracy or reduce variance, we applied bidirectional stepwise selection using stepAIC on a multinomial logistic regression model.

The procedure selected eleven predictors: `S1_Temp, S2_Temp, S3_Temp, S4_Temp, S1_Light, S2_Light, S3_Light, S4_Light, S2_Sound, S5_CO2, S5_CO2_Slope`. These correspond to the most correlated temperature and illumination sensors, as well as the two CO_2-related variables, indicating that thermal, lighting, and air-quality signals contain most of the predictive information in the dataset.

Figure 4 shows how these selected features affect the six models (LDA, Logistic Regression, Naive Bayes, QDA, KNN, and SVM). Performance remains nearly unchanged. Overall, feature selection produces no substantial accuracy gains and does not outperform the full feature set, implying that redundancy among sensors is not harmful for most classifiers and may even slightly benefit models that rely on conditional independence assumptions.

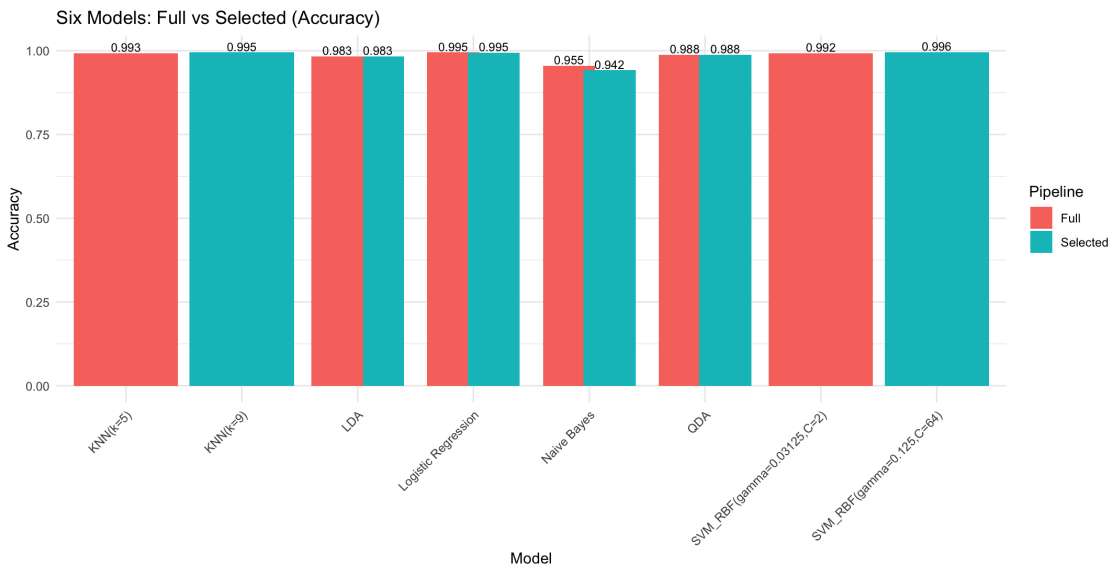

Figure 4: Full vs. Selected Feature Accuracy for Six Models

4.2 PCA Configuration and Effect on Model Performance

Principal Component Analysis (PCA) was applied to the standardized continuous predictors to evaluate whether reducing multicollinearity and compressing the feature space could improve model performance. The optimal number of components was selected separately for each classifier via internal validation. Table 1 summarizes the best-performing PCA configurations.

Table 1: Optimal PCA Components per Model

Model	Best PC Count	Additional Parameter
LDA	15	—
QDA	15	—
Naive Bayes	15	—
Logistic Regression	15	—
KNN	11	$k = 5$
SVM (RBF)	15	—

Most linear and probabilistic models achieved their best results using a high-dimensional PCA space (15 components). KNN benefited from a slightly more compact representation, performing best with 11 components and an internally selected $k = 5$. Because the original dataset contains 16 predictors (14 continuous sensors + 2 PIR binaries), retaining 15 principal components means that PCA offered very limited dimensionality reduction.

Figure 5 compares each model's top PCA configuration against its non-PCA baseline. Across the six classifiers, PCA produced only marginal differences in accuracy (generally within 0.001-0.004). Overall, PCA did not meaningfully change predictive accuracy, indicating that aggressive dimensionality reduction is neither necessary nor beneficial for this dataset.

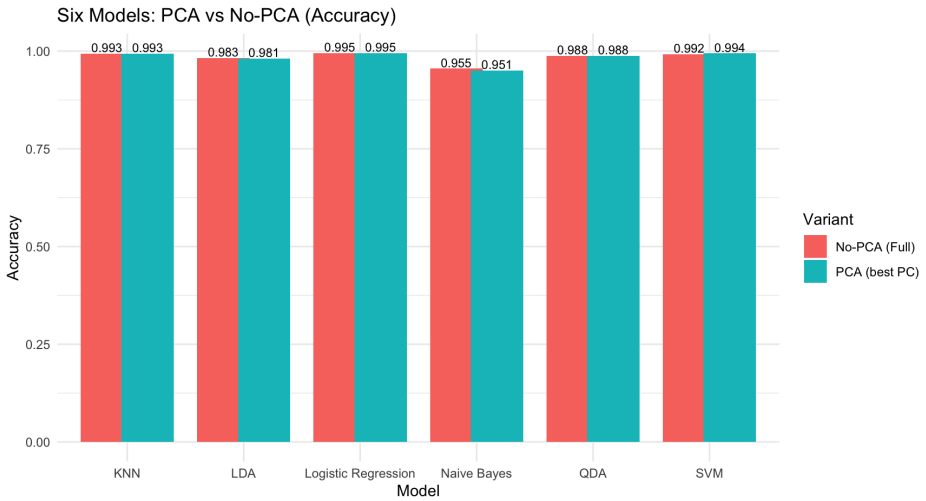

Figure 5: Six Models: PCA vs. No-PCA (Best PC per Model)

4.3 Random Forest and GBM Hyperparameter Tuning

Random Forest hyperparameters were tuned through a grid search over `ntree`, `mtry`, and `nodesize`. The best model achieved an out-of-bag (OOB) error of 0.0022 with:

```
ntree = 600, mtry = 2, nodesize = 1
```

GBM tuning was performed separately for full and selected feature sets using 10-fold cross-validation:

- **GBM using Full Features:** shrinkage = 0.01, depth = 2, CV trees = 2668
- **GBM using Selected Features:** shrinkage = 0.1, depth = 3, CV trees = 140

Both GBM variants achieved very similar cross-validated error of 0.011, indicating that feature selection did not improve predictive accuracy for this model. Although the selected-feature GBM required far fewer trees (140 vs. 2668), the reduction in computational cost did not translate into a meaningful performance gain, suggesting that GBM handles redundant predictors robustly and benefits only modestly from a reduced feature set.

4.4 Overall Model Performance Comparison

Figure 6 presents a ranked comparison of test-set accuracies for all models across all pipelines. Accuracy is uniformly high, ranging from approximately 0.94 (Naive Bayes, Selected) to nearly 0.997 (Random Forest and SVM).

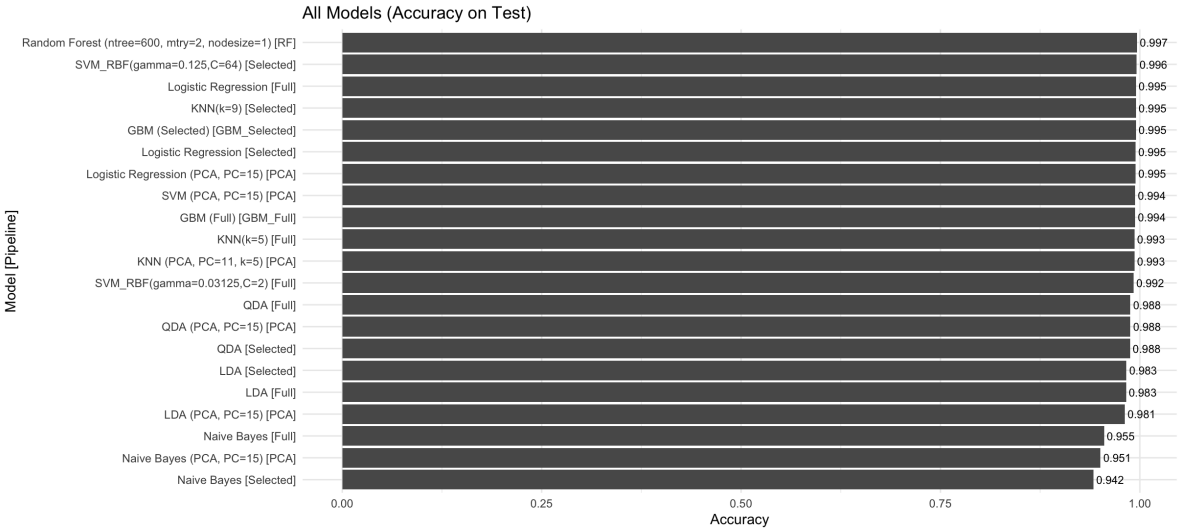

Figure 6: Accuracy of All Models Across Pipelines

Because the performance spread is relatively narrow, accuracy alone is insufficient for distinguishing among top models. This motivates examining additional metrics such as macro-precision, macro-recall, and macro-specificity, which better capture class-level balance and sensitivity to minority occupancy counts.

Table 2 reports all evaluation metrics for each model–pipeline combination. Although overall accuracy remains uniformly strong, the macro-averaged metrics highlight more meaningful differences. Random Forest achieves the strongest overall performance, combining the highest accuracy (0.9965) with excellent macro-precision, macro-recall, and specificity. SVM (Selected) performs comparably well, with balanced macro metrics that slightly exceed those of Logistic Regression. Linear models (LDA and Logistic Regression) remain competitive, consistently exceeding 0.98 accuracy, but show lower macro-recall, reflecting reduced sensi-

tivity to minority occupancy levels. Naive Bayes performs worst across pipelines, primarily due to its independence assumptions.

Table 2: Test Performance of All Models Across All Pipelines

Model	**Accuracy**	**Prec$_{\text{micro}}$**	**Rec$_{\text{micro}}$**	**F1$_{\text{micro}}$**	**Prec$_{\text{macro}}$**	**Rec$_{\text{macro}}$**	**F1$_{\text{macro}}$**	**Spec$_{\text{macro}}$**
Random Forest (Full)	0.9965	0.9965	0.9965	0.9965	0.9898	0.9873	0.9986	0.9986
SVM_RBF (Selected)	0.9961	0.9961	0.9961	0.9961	0.9881	0.9864	0.9872	0.9984
Logistic_Reg (Full)	0.9951	0.9951	0.9951	0.9951	0.9833	0.9825	0.9829	0.9987
KNN (Selected)	0.9951	0.9951	0.9951	0.9951	0.9837	0.9835	0.9836	0.9987
GBM (Selected)	0.9951	0.9951	0.9951	0.9951	0.9841	0.9821	0.9831	0.9982
Logistic_Reg (Selected)	0.9946	0.9946	0.9946	0.9946	0.9814	0.9826	0.9819	0.9985
Logistic_Reg (PCA)	0.9946	0.9946	0.9946	0.9946	0.9813	0.9813	0.9812	0.9985
SVM (PCA)	0.9941	0.9941	0.9941	0.9941	0.9799	0.9835	0.9817	0.9984
GBM (Full)	0.9941	0.9941	0.9941	0.9941	0.9806	0.9832	0.9819	0.9979
KNN (Full)	0.9931	0.9931	0.9931	0.9931	0.9761	0.9798	0.9780	0.9976
KNN (PCA)	0.9931	0.9931	0.9931	0.9931	0.9748	0.9798	0.9773	0.9981
SVM_RBF (Full)	0.9921	0.9921	0.9921	0.9921	0.9746	0.9768	0.9757	0.9974
QDA (Full)	0.9882	0.9882	0.9882	0.9882	0.9619	0.9745	0.9677	0.9963
QDA (PCA)	0.9882	0.9882	0.9882	0.9882	0.9620	0.9759	0.9684	0.9963
QDA (Selected)	0.9877	0.9877	0.9877	0.9877	0.9605	0.9744	0.9668	0.9962
LDA (Selected)	0.9832	0.9832	0.9832	0.9832	0.9576	0.9449	0.9503	0.9900
LDA (Full)	0.9827	0.9827	0.9827	0.9827	0.9572	0.9430	0.9492	0.9894
LDA (PCA)	0.9812	0.9812	0.9812	0.9812	0.9550	0.9423	0.9477	0.9885
Naive Bayes (Full)	0.9551	0.9551	0.9551	0.9551	0.8692	0.8888	0.8776	0.9849
Naive Bayes (PCA)	0.9506	0.9506	0.9506	0.9506	0.8708	0.9245	0.8901	0.9862
Naive Bayes (Selected)	0.9418	0.9418	0.9418	0.9418	0.8527	0.8981	0.8729	0.9754

4.5 Feature Importance Analysis

Because Random Forest achieved the highest test accuracy among all models, we use it as the primary tool for assessing feature importance. Figure 7 reports variable rankings based on both MeanDecreaseAccuracy and MeanDecreaseGini. The results highlight a consistent pattern: lighting sensors (S1-S3 Light) and the two CO_2 features (S5 CO2 and S5 CO2 Slope) are the strongest predictors of occupancy. Temperature sensors follow closely, contributing meaningful but comparatively smaller signal. In contrast, PIR and sound sensors remain less influential, which aligns with the EDA observation that PIR primarily indicates presence rather than precise occupancy counts, and sound levels are more variable and noisy.

These importance patterns reinforce several conclusions from earlier analyses. First, light and CO_2 dynamics contain the clearest discriminative information across occupancy levels. Second, the CO_2 slope feature effectively captures accumulation trends that improve model stability. Finally, the weaker influence of PIR and sound explains why simpler generative models struggled when correlations and heterogeneous variance across sensor types were present.

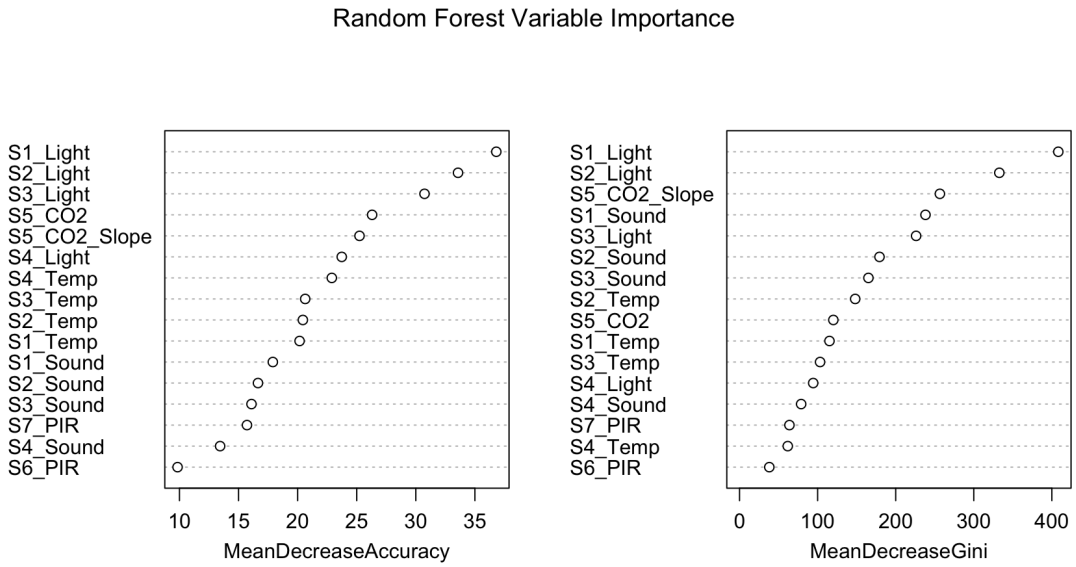

Figure 7: Random Forest Variable Importance

Overall, the feature importance analysis supports the broader finding that environmental sensing, particularly illumination and CO_2 signals, provides a highly reliable basis for data-driven occupancy estimation. Ensemble models such as Random Forest leverage these signals most effectively, achieving near-perfect accuracy with little need for extensive dimensionality reduction.

5 Conclusion

This study investigated supervised learning methods for estimating room occupancy counts (0-3 people) using non-intrusive environmental sensor data. Our analysis addressed the core research questions: (1) whether environmental variables can accurately predict occupancy, (2) which features are most informative, (3) whether PCA provides meaningful benefits, and (4) whether CO_2 dynamics improve prediction.

The results demonstrate that environmental sensing provides exceptionally strong predictive signal for occupancy classification. Nearly all models achieved accuracy above 0.98, with ensemble methods (Random Forest, GBM), SVMs, and regularized linear models performing best. These findings reflect both the controlled nature of the dataset and the strong physical relationships between occupancy and environmental changes such as light intensity, CO_2 accumulation, and temperature. The CO_2 slope feature proved consistently useful, supporting the hypothesis that dynamic air-quality changes capture delayed physiological effects of human presence.

Feature importance analysis further confirmed light and CO_2 as the dominant predictors, with temperature contributing moderately, and PIR and sound playing more limited roles. These conclusions align with prior studies emphasizing CO_2 dynamics and illumination as robust occupancy indicators. While feature selection via stepwise AIC offered a more compact

feature set, its effect on performance was minimal for most models, and occasionally detrimental for generative approaches such as Naive Bayes. This suggests that the full feature set contains redundant but mildly useful variance that some models leverage.

PCA likewise produced negligible differences in model accuracy. Although it reduced multicollinearity and slightly improved performance for KNN, most models achieved their best results with a high-dimensional PCA space (15 components)-essentially retaining almost all of the original information. Given that the full dataset contained 16 predictors, PCA offered minimal dimensionality reduction. The lack of strong PCA benefits indicates that standardized original features already capture the structure needed for accurate classification.

Overall, the study demonstrates that environmental sensor-based occupancy estimation is highly effective with classical machine learning models, even without aggressive dimensionality reduction or complex feature engineering. The findings reinforce the robustness of environmental proxies for occupancy and highlight pathways for efficient real-time deployment using compact feature subsets or faster models.

5.1 Lessons We Have Learned

Several broader insights emerged from this project. First, we found that high predictive performance does not always imply the need for sophisticated model architectures. In this dataset, strong physical signals-illumination, CO_2, and temperature-were sufficiently informative for traditional classifiers to perform near-perfectly. As a result, the marginal gains of techniques such as PCA or aggressive feature selection were limited. This emphasizes the importance of understanding the underlying data-generating mechanisms rather than assuming that algorithmic complexity will yield improvements.

Second, we observed that model families respond differently to correlated features and redundant inputs. Linear and discriminative models were largely unaffected by redundant predictors, while generative methods such as Naive Bayes were more sensitive, experiencing measurable performance drops when features were removed or transformed. This contrast reinforced the conceptual distinctions between modeling approaches and helped justify the inclusion of ensemble and margin-based methods in the comparison.

Third, we learned the value of integrating both static and dynamic environmental indicators. CO_2 slope, although simple to compute, consistently improved performance and provided a richer representation of occupancy-related physiological dynamics. This finding highlights how even modest feature engineering grounded in domain knowledge can materially benefit model performance.

Finally, the process underscored the importance of visualization, cross-validation, and systematic benchmarking. EDA patterns regarding correlation structure, class separation, and feature scales informed preprocessing decisions and prevented unnecessary complexity later in the pipeline. The near-equivalent performance between full, selected, and PCA-transformed models also reinforced that model evaluation must consider both statistical performance and computational efficiency, especially for real-time or embedded applications.

Collectively, these lessons illustrate that effective occupancy modeling depends as much on careful data understanding and feature reasoning as it does on algorithm selection. The

insights gained here can guide future work involving online prediction, temporal models, and deployment in more heterogeneous real-world environments.

Appendix

A Code Listings

Below is the entire R code for reference.

```r
library(ggplot2)
library(dplyr)
library(tidyr)
library(ggcorrplot)
library(corrplot)
library(nnet)          # multinom
library(MASS)          # LDA/QDA + stepAIC
library(e1071)         # naiveBayes + svm + tune.svm
library(class)         # knn
library(randomForest)
library(gbm)
library(purrr)
occ <- read.table(file = "Occupancy_Estimation.csv", sep = ",", header = TRUE)

# EDA
cat("Raw dim:", paste(dim(occ), collapse = " x "), "\n")
print(summary(occ))
df <- {
  tmp <- occ
  tmp <- tmp[, -c(1, 2)]
  tmp$y_fac <- as.factor(tmp$Room_Occupancy_Count)
  tmp$y_num <- tmp$Room_Occupancy_Count
  tmp
}
x_cols <- setdiff(names(df), c("Room_Occupancy_Count", "y_fac", "y_num"))
cat("\nRows x Cols:", nrow(df), "x", ncol(df), "\n\n")
str(df)
cat("\nMissing values per column:\n")
print(sapply(df, function(x) sum(is.na(x))))
cat("\nOriginal occupancy class distribution (factor):\n")
print(table(df$y_fac))
num_df <- df %>% dplyr::select(all_of(x_cols)) %>% dplyr::select(where(is.numeric))
cat("\nNumeric summaries (predictors):\n")
print(summary(num_df))
cor_mat <- cor(num_df, use = "pairwise.complete.obs")
cat("\nCorrelation matrix (numeric predictors):\n")
```

```r
print(round(cor_mat, 3))
corrplot::corrplot(cor_mat, method = "color", type = "upper", tl.cex = 0.7)
title("Correlation Heatmap", line = 1)
## Boxplots
sensors_to_plot <- names(num_df)
df_long <- df %>%
  dplyr::select(all_of(sensors_to_plot), y_fac) %>%
  pivot_longer(cols = -y_fac, names_to = "sensor", values_to = "value")
ggplot(df_long, aes(y_fac, value)) +
  geom_boxplot(outlier.alpha = 0.3) +
  facet_wrap(~ sensor, scales = "free_y") +
  labs(title = "Sensors vs Occupancy Class (Boxplots)", x = "Occupancy Class", y = "Value")
## Scatter
scat_vars <- names(num_df)
df_scat <- df %>%
  dplyr::select(all_of(scat_vars), y_num) %>%
  pivot_longer(cols = -y_num, names_to = "sensor", values_to = "value")
ggplot(df_scat, aes(value, y_num)) +
  geom_point(alpha = 0.3) +
  facet_wrap(~ sensor, scales = "free_x") +
  coord_cartesian(ylim = c(0, 3)) +
  labs(title = "Predictors vs Occupancy", x = "Predictor value", y = "Occupancy")

# Split training/ testing data
unique(occ$S6_PIR)
unique(occ$S7_PIR)
occ <- occ[, -c(1, 2)]
occ[, 1:14] <- scale(occ[, 1:14])
occ$Room_Occupancy_Count <- as.factor(occ$Room_Occupancy_Count)
set.seed(7406)
n_all  <- nrow(occ)
idx_te <- sort(sample(1:n_all, round(0.2 * n_all)))
occ_train <- occ[-idx_te, ]
occ_test  <- occ[idx_te,  ]
cat("Train/Test:", nrow(occ_train), nrow(occ_test), "\n")
X_cols_full <- names(occ_train)[names(occ_train)!="Room_Occupancy_Count"]
y_tr <- occ_train$Room_Occupancy_Count
y_te <- occ_test$Room_Occupancy_Count

# Evaluation metrics helper
compute_metrics <- function(y_true, y_pred) {
```

```r
  y_true <- factor(y_true)
  y_pred <- factor(y_pred, levels = levels(y_true))
  cm <- table(y_true, y_pred)
  acc <- sum(diag(cm)) / sum(cm)
  classes <- levels(y_true)
  prec <- rec <- spec <- f1 <- numeric(length(classes))
  for (i in seq_along(classes)) {
    pos <- classes[i]
    TP <- cm[pos, pos]
    FP <- sum(cm[, pos]) - TP
    FN <- sum(cm[pos, ]) - TP
    TN <- sum(cm) - TP - FP - FN
    prec[i] <- ifelse((TP + FP) > 0, TP / (TP + FP), 0)
    rec[i]  <- ifelse((TP + FN) > 0, TP / (TP + FN), 0)
    spec[i] <- ifelse((TN + FP) > 0, TN / (TN + FP), 0)
    f1[i]   <- ifelse((prec[i] + rec[i]) > 0, 2 * prec[i] * rec[i] / (prec[i] + rec[i]), 0)
  }
  TP_sum <- sum(diag(cm))
  FP_sum <- sum(colSums(cm)) - TP_sum
  FN_sum <- sum(rowSums(cm)) - TP_sum
  prec_micro <- ifelse((TP_sum + FP_sum) > 0, TP_sum / (TP_sum + FP_sum), 0)
  rec_micro  <- ifelse((TP_sum + FN_sum) > 0, TP_sum / (TP_sum + FN_sum), 0)
  f1_micro   <- ifelse((prec_micro + rec_micro) > 0, 2 * prec_micro * rec_micro / (prec_micro + rec_micro), 0)
  data.frame(
    Accuracy          = acc,
    Precision_micro   = prec_micro,
    Recall_micro      = rec_micro,
    F1_micro          = f1_micro,
    Precision_macro   = mean(prec),
    Recall_macro      = mean(rec),
    F1_macro          = mean(f1),
    Specificity_macro = mean(spec)
  )
}
add_row <- function(tab, model_name, pipeline, y_true, y_pred) {
  met <- compute_metrics(y_true, y_pred)
  dplyr::bind_rows(tab, cbind(Model = model_name, Pipeline = pipeline, met))
}

# Training/Evaluting LDA, QDA, Naive Bayes, Logistic, KNN, SVM
fit_predict_six <- function(Xtr, ytr, Xte, seed = 7406, pipeline_tag = "Full", search_knn = TRUE) {
```

```r
set.seed(seed)
out <- list(rows = list())
## LDA
m_lda <- lda(ytr ~ ., data = data.frame(Xtr, ytr = ytr))
p_lda <- predict(m_lda, newdata = data.frame(Xte))$class
out$rows[["LDA"]] <- list(name="LDA", pred=p_lda)
## QDA
m_qda <- qda(ytr ~ ., data = data.frame(Xtr, ytr = ytr))
p_qda <- predict(m_qda, newdata = data.frame(Xte))$class
out$rows[["QDA"]] <- list(name="QDA", pred=p_qda)
## Naive Bayes
m_nb  <- naiveBayes(ytr ~ ., data = data.frame(Xtr, ytr = ytr))
p_nb  <- predict(m_nb, newdata = data.frame(Xte))
out$rows[["Naive Bayes"]] <- list(name="Naive Bayes", pred=p_nb)
## Multinomial Logistic
m_log <- multinom(ytr ~ ., data = data.frame(Xtr, ytr = ytr),
   trace = FALSE)
p_log <- predict(m_log, newdata = data.frame(Xte))
out$rows[["Logistic Regression"]] <- list(name="Logistic
   Regression", pred=p_log)
## KNN
if (isTRUE(search_knn)) {
  set.seed(seed)
  n_tr <- nrow(Xtr); val_n <- round(0.2*n_tr)
  idx  <- sort(sample(1:n_tr, val_n))
  tr_b <- Xtr[-idx, , drop=FALSE]; yt_b <- ytr[-idx]
  va_b <- Xtr[ idx, , drop=FALSE]; yv_b <- ytr[ idx]
  kk   <- c(1,3,5,7,9)
  te   <- sapply(kk, function(k){
    pv <- knn(tr_b, va_b, yt_b, k = k)
    mean(pv != yv_b)
  })
  k_best <- kk[which.min(te)]
} else k_best <- 3
p_knn <- knn(Xtr, Xte, ytr, k = k_best)
out$rows[["KNN"]] <- list(name=paste0("KNN(k=",k_best,")"), pred=
   p_knn)
# SVM (RBF)
g_grid <- 2^seq(-6, -1, by=1)
c_grid <- 2^seq(-1,  6, by=1)
tune_res <- tune.svm(x = as.matrix(Xtr), y = ytr, kernel="radial",
                     gamma = g_grid, cost = c_grid, probability =
                        FALSE)
m_svm <- tune_res$best.model
p_svm <- predict(m_svm, as.matrix(Xte))
out$rows[["SVM"]] <- list(name=paste0("SVM_RBF(gamma=",m_svm$gamma
   ,",C=",m_svm$cost,")"), pred=p_svm)
```

```r
  # collect to table
  tab <- data.frame()
  for (k in names(out$rows)) {
    tab <- add_row(tab, out$rows[[k]]$name, pipeline_tag, y_te, out$rows[[k]]$pred)
  }
  attr(tab, "models") <- list(lda=m_lda, qda=m_qda, nb=m_nb, log=m_log,
                              knn_k = k_best, svm=m_svm)
  tab
}

# StepAIC feature selection
mod_log_full_for_fs <- multinom(Room_Occupancy_Count ~ ., data = occ_train, trace = FALSE)
mod_log_step <- stepAIC(mod_log_full_for_fs, direction = "both", trace = FALSE)
sel_terms <- attr(terms(formula(mod_log_step)), "term.labels")
cat("\n[StepAIC selected predictors]\n"); print(sel_terms)
sel_cols <- intersect(sel_terms, X_cols_full)

# Pipeline #1: 6 models on full features
Xtr_full <- occ_train[, X_cols_full, drop=FALSE]
Xte_full <- occ_test [, X_cols_full, drop=FALSE]
results_tbl <- fit_predict_six(Xtr_full, y_tr, Xte_full, seed=7406, pipeline_tag="Full")
# Pipeline #2: 6 models on selected features
if (length(sel_cols) >= 1) {
  Xtr_sel <- occ_train[, sel_cols, drop=FALSE]
  Xte_sel <- occ_test [, sel_cols, drop=FALSE]
  results_tbl_sel <- fit_predict_six(Xtr_sel, y_tr, Xte_sel, seed=7406, pipeline_tag="Selected")
  results_tbl <- dplyr::bind_rows(results_tbl, results_tbl_sel)
} else {
  cat("\n[Selected pipeline] No predictors selected by stepAIC; skipped.\n")
}
# Pipeline #3: 6 models on PCA
pc_grid <- c(5, 9, 11, 15)
pca_full <- prcomp(Xtr_full, scale. = FALSE)
## Return the best PC
pick_best_pc_for_model <- function(model_tag){
  set.seed(7406)
  n_tr <- nrow(Xtr_full); val_n <- round(0.2*n_tr)
  idx  <- sort(sample(1:n_tr, val_n))
  yt_b <- y_tr[-idx]; yv_b <- y_tr[idx]
```

```r
best <- list(pc = NULL, loss = Inf, k = NULL)  # k used only for KNN
for (pc in pc_grid[pc_grid <= ncol(pca_full$x)]) {
  tr_pca <- data.frame(pca_full$x[-idx, 1:pc, drop=FALSE])
  va_pca <- data.frame(pca_full$x[ idx, 1:pc, drop=FALSE])
  if (model_tag == "LDA") {
    mdl <- lda(yt_b ~ ., data = data.frame(tr_pca, yt_b=yt_b))
    pr  <- predict(mdl, newdata = va_pca)$class
    loss <- mean(pr != yv_b)
    if (loss < best$loss) best <- list(pc=pc, loss=loss, k=NULL)
  } else if (model_tag == "QDA") {
    mdl <- qda(yt_b ~ ., data = data.frame(tr_pca, yt_b=yt_b))
    pr  <- predict(mdl, newdata = va_pca)$class
    loss <- mean(pr != yv_b)
    if (loss < best$loss) best <- list(pc=pc, loss=loss, k=NULL)
  } else if (model_tag == "Naive Bayes") {
    mdl <- naiveBayes(yt_b ~ ., data = data.frame(tr_pca, yt_b=yt_b))
    pr  <- predict(mdl, newdata = va_pca)
    loss <- mean(pr != yv_b)
    if (loss < best$loss) best <- list(pc=pc, loss=loss, k=NULL)
  } else if (model_tag == "Logistic Regression") {
    mdl <- multinom(yt_b ~ ., data = data.frame(tr_pca, yt_b=yt_b), trace = FALSE)
    pr  <- predict(mdl, newdata = va_pca)
    loss <- mean(pr != yv_b)
    if (loss < best$loss) best <- list(pc=pc, loss=loss, k=NULL)
  } else if (model_tag == "KNN") {
    # NEW: jointly search k in {1,3,5,7,9} for each PC, keep best (PC,k) pair overall
    kk <- c(1,3,5,7,9)
    te_vec <- sapply(kk, function(k) {
      pv <- knn(tr_pca, va_pca, yt_b, k = k)
      mean(pv != yv_b)
    })
    k_here <- kk[which.min(te_vec)]
    loss   <- min(te_vec)
    if (loss < best$loss) best <- list(pc=pc, loss=loss, k=k_here)
  } else if (model_tag == "SVM") {
    g_grid <- 2^seq(-6, -1, by=1)
    c_grid <- 2^seq(-1,  6, by=1)
    tune_res <- tune.svm(x = as.matrix(tr_pca), y = yt_b, kernel="radial",
                         gamma = g_grid, cost = c_grid, probability = FALSE)
    pr   <- predict(tune_res$best.model, as.matrix(va_pca))
    loss <- mean(pr != yv_b)
```

```r
      if (loss < best$loss) best <- list(pc=pc, loss=loss, k=NULL)
    }
  }
  best
}
six_names <- c("LDA","QDA","Naive Bayes","Logistic Regression","KNN"
    ,"SVM")
# Store best configs for each model
best_cfg_map <- setNames(vector("list", length(six_names)),
    six_names)
for (nm in six_names) best_cfg_map[[nm]] <- pick_best_pc_for_model(
    nm)
cat("\n[PCA best config per model]\n")
print(lapply(best_cfg_map, function(z) { if (is.null(z$k)) list(PC=
    z$pc) else list(PC=z$pc, k=z$k) }))
# Train each model and test its performance
res_pca <- data.frame()
for (nm in six_names) {
  cfg <- best_cfg_map[[nm]]
  pc  <- cfg$pc
  tr_p <- data.frame(pca_full$x[, 1:pc, drop=FALSE])
  te_p <- data.frame(predict(pca_full, newdata = Xte_full)[, 1:pc,
      drop=FALSE])
  if (nm=="LDA") {
    mdl <- lda(y_tr ~ ., data = data.frame(tr_p, y_tr=y_tr))
    pr  <- predict(mdl, newdata = te_p)$class
    label <- sprintf("LDA (PCA, PC=%d)", pc)
  } else if (nm=="QDA") {
    mdl <- qda(y_tr ~ ., data = data.frame(tr_p, y_tr=y_tr))
    pr  <- predict(mdl, newdata = te_p)$class
    label <- sprintf("QDA (PCA, PC=%d)", pc)
  } else if (nm=="Naive Bayes") {
    mdl <- naiveBayes(y_tr ~ ., data = data.frame(tr_p, y_tr=y_tr))
    pr  <- predict(mdl, newdata = te_p)
    label <- sprintf("Naive Bayes (PCA, PC=%d)", pc)
  } else if (nm=="Logistic Regression") {
    mdl <- multinom(y_tr ~ ., data = data.frame(tr_p, y_tr=y_tr),
        trace = FALSE)
    pr  <- predict(mdl, newdata = te_p)
    label <- sprintf("Logistic Regression (PCA, PC=%d)", pc)
  } else if (nm=="KNN") {
    k_best <- cfg$k
    pr  <- knn(tr_p, te_p, y_tr, k = k_best)
    label <- sprintf("KNN (PCA, PC=%d, k=%d)", pc, k_best)
  } else if (nm=="SVM") {
    g_grid <- 2^seq(-6, -1, by=1)
    c_grid <- 2^seq(-1,  6, by=1)
```

```r
    tune_res <- tune.svm(x = as.matrix(tr_p), y = y_tr, kernel="radial",
                         gamma = g_grid, cost = c_grid, probability = FALSE)
    mdl <- tune_res$best.model
    pr  <- predict(mdl, as.matrix(te_p))
    label <- sprintf("SVM (PCA, PC=%d)", pc)
  }
  res_pca <- add_row(res_pca, label, "PCA", y_te, pr)
}
results_tbl <- dplyr::bind_rows(results_tbl, res_pca)

# Random forest
set.seed(7406)
ntree_grid    <- c(400, 500, 600)
nodesize_grid <- c(1, 2, 3)
mtry_grid     <- c(1, 2, 3, 4, 5)
grid_rf <- expand.grid(ntree = ntree_grid, nodesize = nodesize_grid, mtry = mtry_grid)
best_oob <- Inf
best_par <- NULL
for (i in 1:nrow(grid_rf)) {
  rf_fit <- randomForest(Room_Occupancy_Count ~ ., data = occ_train,
                         ntree = grid_rf$ntree[i],
                         mtry = grid_rf$mtry[i],
                         nodesize = grid_rf$nodesize[i])
  oob <- rf_fit$err.rate[nrow(rf_fit$err.rate), "OOB"]
  if (oob < best_oob) { best_oob <- oob; best_par <- grid_rf[i, ] }
}
cat("\n[RF] best OOB =", best_oob, "with", paste(names(best_par), "=", as.numeric(best_par), collapse = ", "), "\n")
set.seed(7406)
rf_best <- randomForest(Room_Occupancy_Count ~ ., data = occ_train,
                        ntree = best_par$ntree, mtry = best_par$mtry,
                        nodesize = best_par$nodesize, importance = TRUE)
pred_rf <- predict(rf_best, occ_test, type = "class")
results_tbl <- add_row(results_tbl,
                       sprintf("Random Forest (ntree=%d, mtry=%d, nodesize=%d)",
                               best_par$ntree, best_par$mtry, best_par$nodesize),
                       "RF", y_te, pred_rf)

# GBM with full & selected features
set.seed(7406)
```

```r
shrink_grid <- c(0.01, 0.05, 0.1)
depth_grid  <- c(1, 2, 3)
tune_gbm_multiclass <- function(formula_str, dat_tr){
  best_cv_err <- Inf; best_par <- NULL; best_iter <- NULL
  for (sh in shrink_grid) {
    for (dp in depth_grid) {
      gbm_fit <- gbm(as.formula(formula_str),
                     data = dat_tr,
                     distribution = "multinomial",
                     n.trees = 5000,
                     shrinkage = sh,
                     interaction.depth = dp,
                     cv.folds = 10, verbose = FALSE)
      iter_cv <- gbm.perf(gbm_fit, method = "cv", plot.it = FALSE)
      cv_err  <- min(gbm_fit$cv.error)
      if (cv_err < best_cv_err) {
        best_cv_err <- cv_err; best_par <- c(shrinkage=sh, depth=dp); best_iter <- iter_cv
      }
    }
  }
  list(cv_err=best_cv_err, par=best_par, iter=best_iter)
}

## GBM with full features
gbm_tuned_full <- tune_gbm_multiclass("Room_Occupancy_Count ~ .", occ_train)
cat("\n[GBM full] best cv err =", gbm_tuned_full$cv_err,
    " shrinkage =", gbm_tuned_full$par["shrinkage"],
    " depth =", gbm_tuned_full$par["depth"],
    " trees =", gbm_tuned_full$iter, "\n")
set.seed(7406)
gbm_full <- gbm(Room_Occupancy_Count ~ ., data = occ_train, distribution = "multinomial",
                n.trees = as.integer(gbm_tuned_full$iter),
                shrinkage = as.numeric(gbm_tuned_full$par["shrinkage"]),
                interaction.depth = as.integer(gbm_tuned_full$par["depth"]),
                cv.folds = 0, verbose = FALSE)
prob_full <- predict(gbm_full, occ_test, n.trees = as.integer(gbm_tuned_full$iter), type = "response")
pred_gbm_full <- factor(apply(prob_full, 1, which.max) - 1, levels = levels(y_te))
results_tbl <- add_row(results_tbl, "GBM (Full)", "GBM_Full", y_te, pred_gbm_full)

```

```r
## GBM with selected features
if (length(sel_cols) >= 1) {
  form_sel <- paste("Room_Occupancy_Count ~", paste(sel_cols, collapse = "+"))
  gbm_tuned_sel <- tune_gbm_multiclass(form_sel, occ_train)
  cat("\n[GBM selected] best cv err =", gbm_tuned_sel$cv_err,
      " shrinkage =", gbm_tuned_sel$par["shrinkage"],
      " depth =", gbm_tuned_sel$par["depth"],
      " trees =", gbm_tuned_sel$iter, "\n")
  set.seed(7406)
  gbm_sel <- gbm(as.formula(form_sel), data = occ_train, distribution = "multinomial",
                 n.trees = as.integer(gbm_tuned_sel$iter),
                 shrinkage = as.numeric(gbm_tuned_sel$par["shrinkage"]),
                 interaction.depth = as.integer(gbm_tuned_sel$par["depth"]),
                 cv.folds = 0, verbose = FALSE)
  prob_sel <- predict(gbm_sel, occ_test, n.trees = as.integer(gbm_tuned_sel$iter), type = "response")
  pred_gbm_sel <- factor(apply(prob_sel, 1, which.max) - 1, levels = levels(y_te))
  results_tbl <- add_row(results_tbl, "GBM (Selected)", "GBM_Selected", y_te, pred_gbm_sel)
}

# Final results
results_tbl <- results_tbl %>%
  mutate(Model = as.character(Model),
         Pipeline = as.character(Pipeline)) %>%
  relocate(Model, Pipeline) %>%
  arrange(desc(Accuracy))
cat("\n============== Model Metrics (Test) ==============\n")
print(results_tbl, row.names = FALSE)

## All models accuracy chart
ggplot(results_tbl, aes(x = reorder(paste0(Model," [",Pipeline,"]"), Accuracy), y = Accuracy)) +
  geom_col() +
  coord_flip() +
  geom_text(aes(label = sprintf("%.3f", Accuracy)), hjust = -0.1, size = 3) +
  labs(title = "All Models (Accuracy on Test)", x = "Model [Pipeline]", y = "Accuracy") +
  theme_minimal()
## 6 models: Full vs Selected comparison
comp_FS <- results_tbl %>%
```

```r
  filter(Pipeline %in% c("Full","Selected"),
         grepl("^(LDA|QDA|Naive Bayes|Logistic Regression|KNN|SVM)", Model)) %>%
  select(Model, Pipeline, Accuracy)
ggplot(comp_FS, aes(x = Model, y = Accuracy, fill = Pipeline)) +
  geom_col(position = position_dodge(width = 0.7)) +
  geom_text(aes(label = sprintf("%.3f", Accuracy)), position = position_dodge(width = 0.7),
            vjust = -0.2, size = 3) +
  labs(title = "Six Models: Full vs Selected (Accuracy)", x = "Model", y = "Accuracy") +
  theme_minimal() + theme(axis.text.x = element_text(angle = 45, hjust = 1))
## 6 models: PCA vs No-PCA comparison
comp_PCA <- results_tbl %>%
  mutate(Base = case_when(
    grepl("^LDA", Model) ~ "LDA",
    grepl("^QDA", Model) ~ "QDA",
    grepl("^Naive Bayes", Model) ~ "Naive Bayes",
    grepl("^Logistic Regression", Model) ~ "Logistic Regression",
    grepl("^KNN", Model) ~ "KNN",
    grepl("^SVM", Model) ~ "SVM",
    TRUE ~ NA_character_
  )) %>%
  filter(Base %in% six_names, Pipeline %in% c("Full","PCA")) %>%
  mutate(Variant = ifelse(Pipeline=="PCA","PCA (best PC)","No-PCA (Full)")) %>%
  group_by(Base, Variant) %>%
  slice_max(order_by = Accuracy, n = 1, with_ties = FALSE) %>%
  ungroup() %>%
  rename(ModelBase = Base)
ggplot(comp_PCA, aes(x = ModelBase, y = Accuracy, fill = Variant)) +
  geom_col(position = position_dodge(width = 0.7)) +
  geom_text(aes(label = sprintf("%.3f", Accuracy)), position = position_dodge(width = 0.7),
            vjust = -0.2, size = 3) +
  labs(title = "Six Models: PCA vs No-PCA (Accuracy)", x = "Model", y = "Accuracy") +
  theme_minimal()
## Display RF feature importance
rf_imp <- as.data.frame(importance(rf_best))
rf_imp$Variable <- rownames(rf_imp)
rf_imp <- rf_imp %>% arrange(desc(MeanDecreaseAccuracy))
cat("\n[Random Forest Variable Importance by MeanDecreaseAccuracy]\n")
print(rf_imp)
varImpPlot(rf_best, main = "Random Forest Variable Importance")
```

References

[1] Luis M. Candanedo and Veronique Feldheim. *Accurate occupancy detection of an office room from light, temperature, humidity and CO2 measurements using statistical learning models*. Energy and Buildings, Elsevier. Accessed: 10-05-2025. 2016. URL: https://www.sciencedirect.com/science/article/abs/pii/S0378778815304357.

[2] Jinwoo Kim, Seunghyun Lee, and Chang Park. *Estimation of occupancy using IoT sensors and carbon concentration models*. Sensors and Actuators A: Physical. Accessed: 10-05-2025. 2023. URL: https://pmc.ncbi.nlm.nih.gov/articles/PMC9860618.

[3] UCI Machine Learning Repository. *Room Occupancy Estimation Data Set*. University of California, Irvine Machine Learning Repository. Accessed: 10-05-2025. 2024. URL: https://archive.ics.uci.edu/dataset/864/room+occupancy+estimation.

[4] Wei Zhang, Han Wang, and Jian Liu. *A review on occupancy prediction through machine learning*. Renewable and Sustainable Energy Reviews. Accessed: 10-05-2025. 2022. URL: https://www.sciencedirect.com/science/article/pii/S1364032122005937.